



신용카드 사용자 연체 예측

겨울방학 프로젝트

YB 1조 김재경, 유재은, 이우림, 전예원

목차

1. 프로젝트 소개

2. EDA 및 전처리

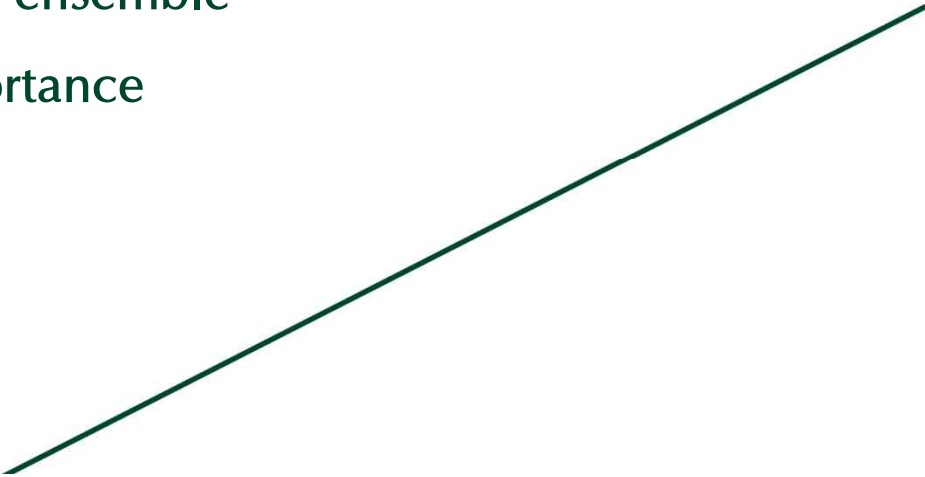
- 1) EDA
- 2) 전처리
- 3) 파생변수 생성
- 4) 범주형 칼럼
- 5) 불필요한 변수 제거

3. 모델링

- 1) Catboost
- 2) Optuna 하이퍼파라미터 튜닝
- 3) 평가지표: Logloss
- 4) 5-fold 7-seed ensemble
- 5) feature importance

4. 제출 결과

5. Q&A



1. 프로젝트 소개

신용카드 사용자 연체 예측 AI 경진대회

- 목적: 신용카드 사용자 데이터를 보고 사용자의 대금 연체 정도를 예측하는 알고리즘 개발



2-1. EDA

train.csv

train 데이터 : 신용카드 사용자들의 개인 신상정보

credit 열 포함

train.shape : (26457, 20)

test.csv

test 데이터 : 신용카드 사용자들의 개인 신상정보

credit 열 미포함

test.shape : (10000, 19)

index

gender: 성별

car: 차량 소유 여부

realty: 부동산 소유 여부

child_num: 자녀 수

income_total: 연간 소득

income_type: 소득 분류

edu_type: 교육 수준

family_type: 결혼 여부

house_type: 생활 방식

DAYS_BIRTH: 출생일

DAYS_EMPLOYED: 업무 시작일

FLAG_MOBIL: 핸드폰 소유 여부

work_phone: 업무용 전화 소유 여부

phone: 전화 소유 여부

email: 이메일 소유 여부

occyp_type: 직업 유형

family_size: 가족 규모

begin_month: 신용카드 발급 월

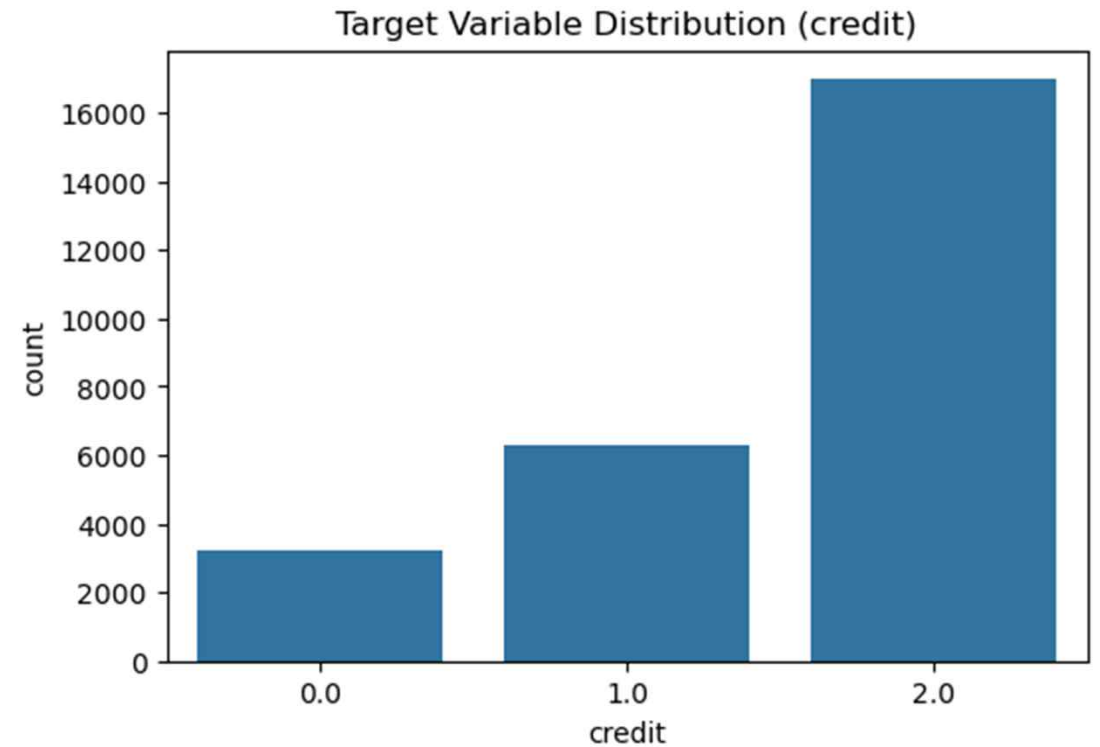
credit: 사용자의 신용카드 대금 연체를 기준으로 한 신용도

=> 낮을 수록 높은 신용의 신용카드 사용자를 의미함

2-1. EDA

타깃 변수 분포

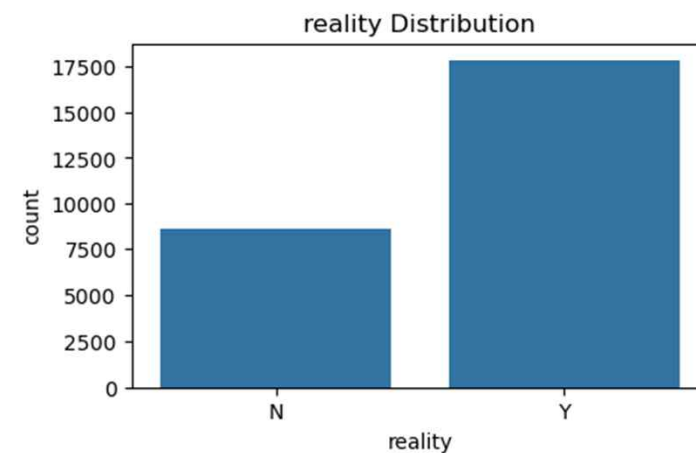
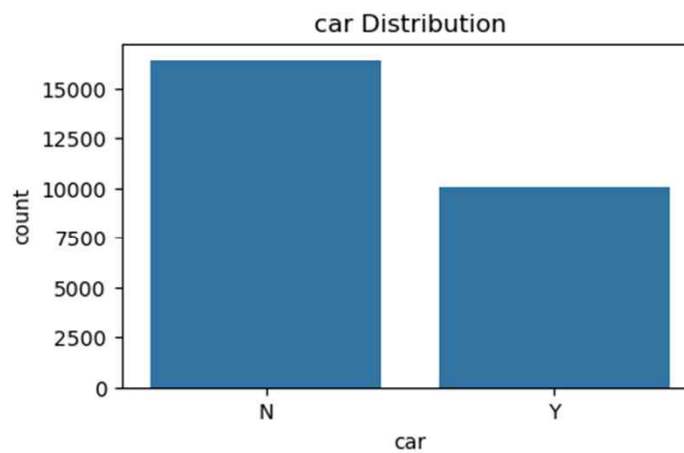
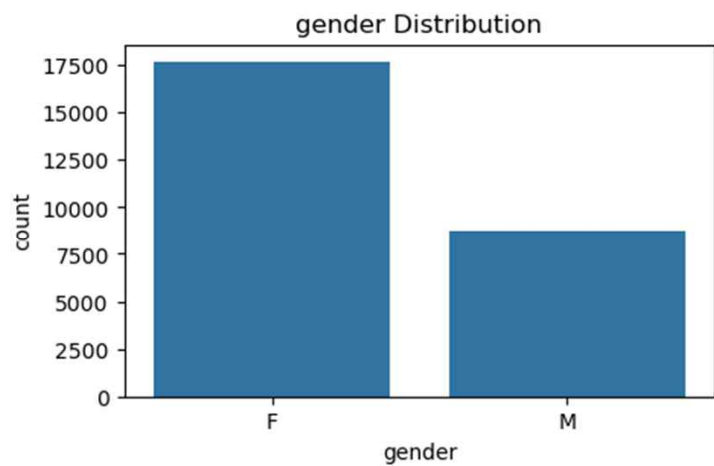
타깃 변수는 클래스 간 불균형이 존재하여
정확도보다 확률 기반 지표(LogLoss)를 사용함



2-1. EDA

범주형 변수 분포

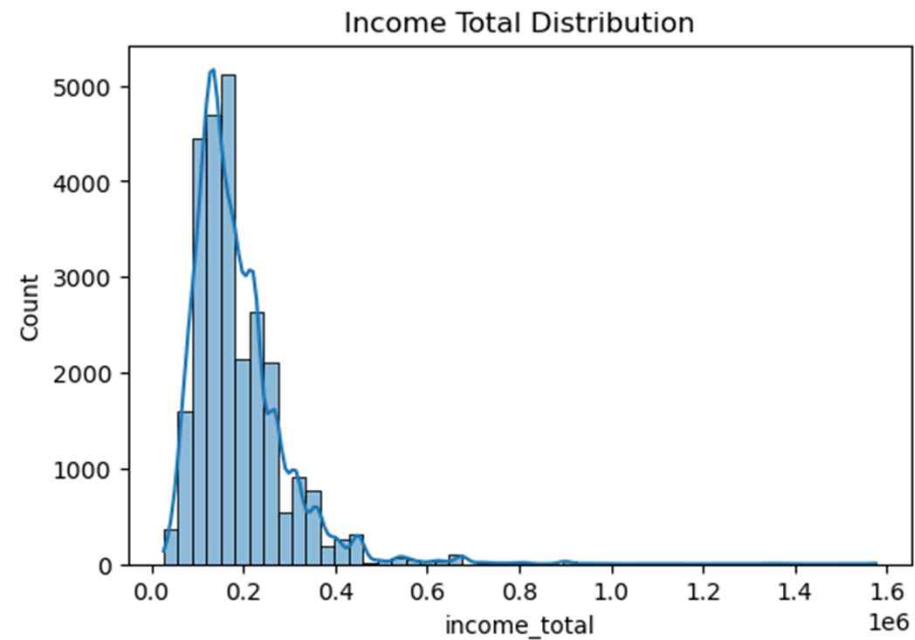
다수의 범주형 변수가 소수 카테고리에 집중되어 있음



2-1. EDA

Income_total 분포

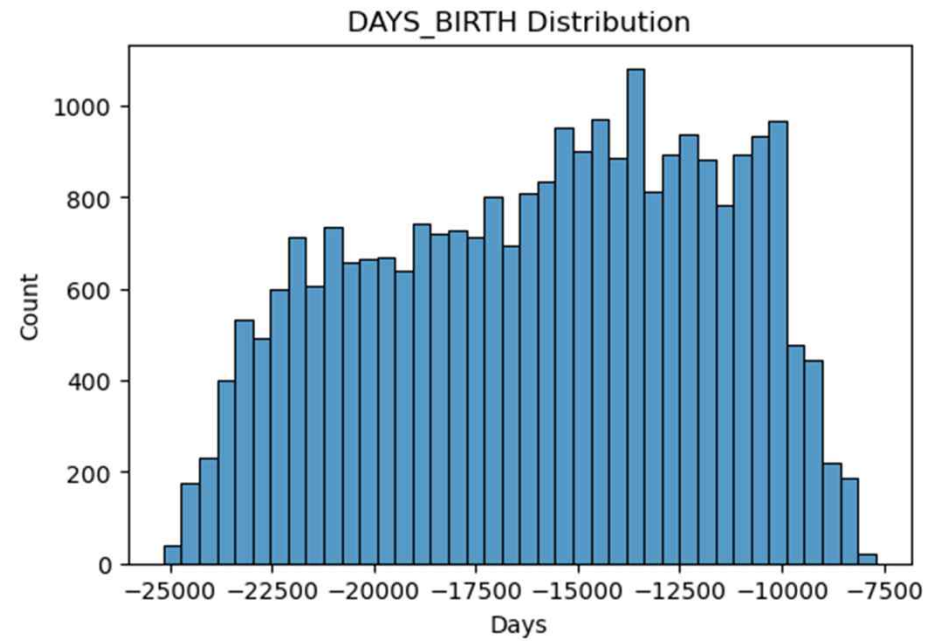
소득 변수는 우측으로 치우친 분포를 가지며 트리 기반 모델에 적합한 형태임



2-1. EDA

DAYS_BIRTH 분포

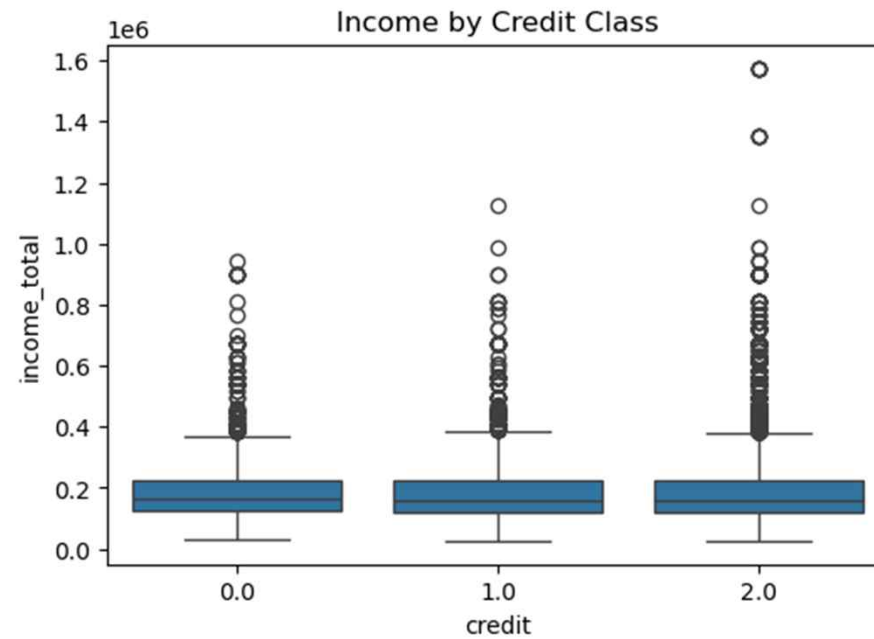
DAYS_BIRTH는 출생일 기준으로 음수 값이며 값이 작을수록 고령을 의미함



2-1. EDA

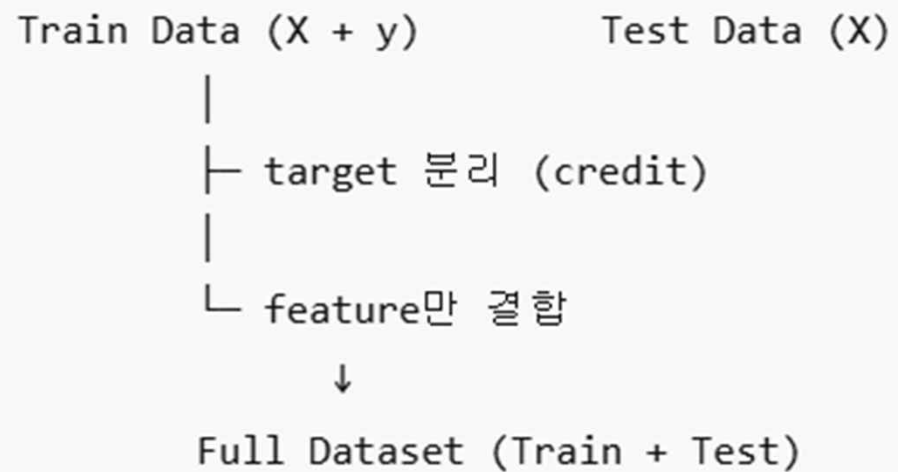
타깃별 수치 변수 비교

연체 등급에 따라 소득 분포 차이가 일부 관찰됨



2-2. 전처리

전처리 단계 ① : 타깃 분리 및 데이터 통합



2-3. 파생변수 생성: user_id

user_id 생성

- 개인 특성 칼럼들을 조합해 가상 ID 생성
- 동일 인물이 여러 행으로 존재하는 구조 해결
- 이후 사용자 단위 집계 피처 생성의 기준이 됨
- 모델이 '행'이 아니라 '사람'을 학습하도록 구조 개선

```
id_cols = [  
    'gender', 'car', 'reality', 'child_num', 'income_total',  
    'income_type', 'edu_type', 'family_type', 'house_type',  
    'DAYS_BIRTH', 'DAYS_EMPLOYED',  
    'work_phone', 'phone', 'email', 'occyp_type'  
]  
  
full['user_id'] = full[id_cols].astype(str).agg('_'.join, axis=1)
```

2-3. 파생변수 생성: 카드 활동 파생 변수

사용자의 카드 이용 이력을 기간, 지속성, 패턴 안정성으로 수치화한 행동 기반 피처 생성

데이터 수집 당시(0)부터
역으로 셈한 값이기 때문에 -를 붙임

```
full['begin_month'] = -full['begin_month']

full['user_card_cnt'] = full.groupby('user_id')['user_id'].transform('count')
full['user_begin_min'] = full.groupby('user_id')['begin_month'].transform('min')
full['user_begin_max'] = full.groupby('user_id')['begin_month'].transform('max')

full['user_begin_range'] = full['user_begin_max'] - full['user_begin_min']
full['user_begin_mean'] = full.groupby('user_id')['begin_month'].transform('mean')
full['user_begin_std'] = (
    full.groupby('user_id')['begin_month'].transform('std').fillna(0)
)

full['months_active'] = full['user_card_cnt']
full['card_age'] = full['user_begin_max'] - full['begin_month']
full['is_new_user'] = (full['months_active'] < 3).astype(int)
```

모든 계산이 유저 단위
-> 사용자 기준 집계

2-3. 파생변수 생성: 카드 활동 파생 변수

변수	계산 방식	의미
begin_month	-begin_month	과거일수록 값 커지도록 시간 방향 정규화
user_card_cnt	사용자별 행 개수	카드 사용 개월 수
user_begin_min	사용자 최소 begin_month	가장 최근 사용 시점
user_begin_max	사용자 최대 begin_month	가장 오래된 사용 시점
user_begin_range	max - min	활동 기간 범위
user_begin_mean	평균 begin_month	활동이 몰린 시기
user_begin_std	표준편차	사용 시점 변동성
months_active	user_card_cnt 동일	활동 기간 길이
card_age	max - 현재 begin_month	이 기록이 전체 이력 중 어디쯤인지
is_new_user	months_active < 3	신규 사용자 여부

2-3. 파생변수 생성: 인구, 소득 특성 정규화 및 비율 변수 생성

```
full['age'] = -full['DAYS_BIRTH'] / 365.25
full['employment_years'] = full['DAYS_EMPLOYED'].apply(
    lambda x: -x / 365.25 if x < 0 else 0
)

full['income_per_person'] = full['income_total'] / (full['family_size'] + 1)
full['log_income'] = np.log1p(full['income_total'])
full['log_income_per_person'] = np.log1p(full['income_per_person'])

full['age_emp_ratio'] = full['age'] / (full['employment_years'] + 1)
full['income_begin_ratio'] = full['income_total'] / (full['begin_month'] + 1)
```

수치형 변수의 스케일 조정, 로그 변환, 상호작용 비율 변수를 통해 경제적 특성을 구조화

2-3. 파생변수 생성: 인구, 소득 특성 정규화 및 비율 변수 생성

변수	계산 방식	의미
age	$-\text{DAYS_BIRTH} / 365.25$	나이(연 단위)
employment_years	근무일수 $\rightarrow$ 연 단위	근속 연수
income_per_person	소득 / (가족수 + 1)	1인당 실질 소득
log_income	$\text{Log}(\text{소득})$	소득 스케일 축소
log_income_per_person	1인당 소득 로그	소득 분포 안정화
age_emp_ratio	나이 / 근속연수	인생 대비 직업 안정도
income_begin_ratio	소득 / 카드사용기간	소득 대비 카드 활동 기간

2-4. 범주형 칼럼

```
cat_cols = [  
    'gender', 'car', 'reality',  
    'income_type', 'edu_type', 'family_type',  
    'house_type', 'occyp_type',  
    'user_id'  
]  
  
# object가 아닌데 범주 의미인 건 문자열로 캐스팅  
for col in cat_cols:  
    full[col] = full[col].astype(str)
```

별도의 인코딩을 수행하지 않고 문자열 형태로 유지

2-5. 불필요한 변수 제거

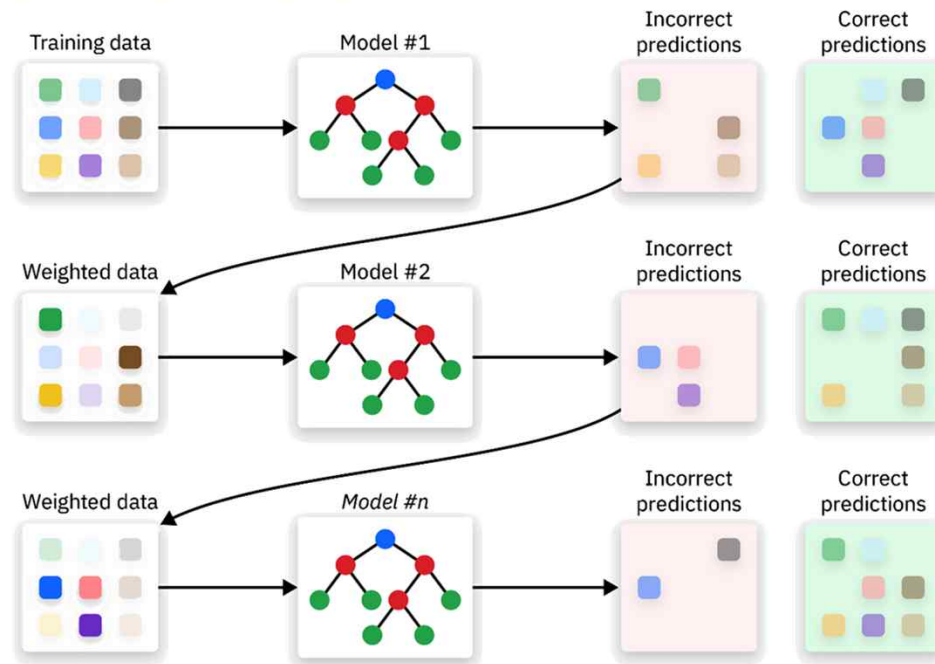
```
drop_cols = [  
    'index',  
    'DAYS_BIRTH',  
    'DAYS_EMPLOYED',  
    'FLAG_MOBIL'  
]  
full = full.drop(columns=[c for c in drop_cols if c in full.columns])
```

나이 및 근속연수 파생 후 원본 변수와 모델에 의미 없는 식별성 칼럼 제거
->학습 효율과 모델 안정성 높임

3-1. CatBoost

Gradient Boosting 기반의 트리 모델로, 범주형 변수 처리에 특화된 알고리즘
CatBoost와 다른 GBDT의 차이점

- Categorical 자동 처리: 정보 손실 감소
- Ordered Boosting: Target leakage 방지
- 과적합에 강함: 랜덤성+정규화, 검증 성능 안정



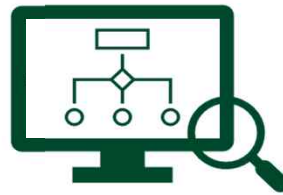
3-1. CatBoost

CatBoost 선정 이유

- 범주형 변수 많음 -> 자동 처리 강점 활용
- user_id 존재 -> 범주 개수가 많은 변수 처리에 강점
- 사용자 행동 패턴 중요 -> 트리 기반 패턴 학습 강점
- 신용 등급 분류 문제 -> MultiClass에 적합



다양한 고객 정보 및
활동 데이터

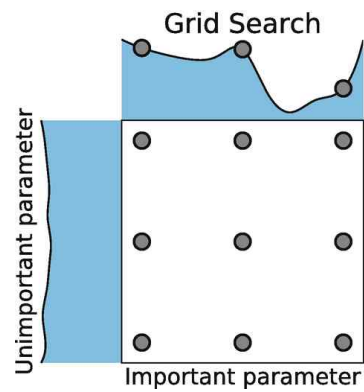


범주형 처리에 특화된 학습

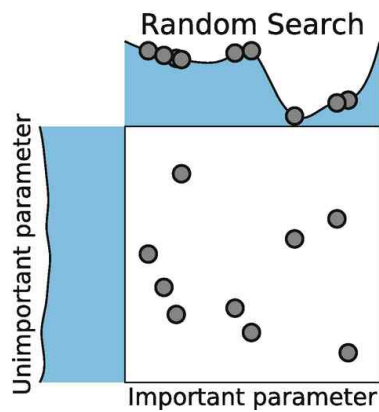


정확한 신용카드 대금
연체 정도 예측

3-2. Optuna 하이퍼파라미터 튜닝

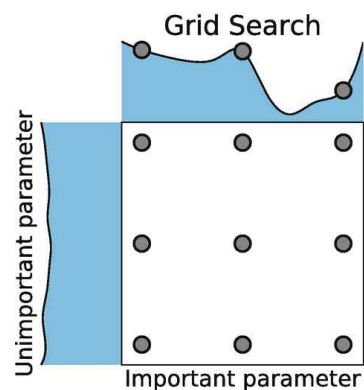


- Grid Search: 느림

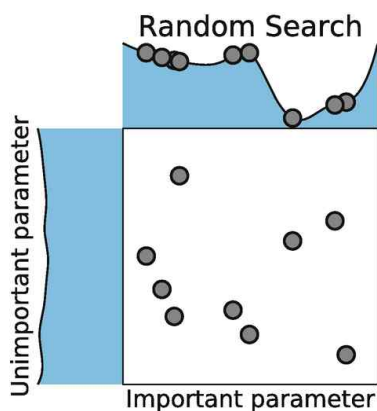


- Random Search:
운에 의존하는 경향

3-2. Optuna 하이퍼파라미터 튜닝



- Grid Search: 느림



- Random Search: 운에 의존하는 경향



베이지안 최적화 알고리즘을 사용해
과거의 시도 결과를 바탕으로
성능이 잘 나올 것 같은 파라미터 범위를 추적함



- **Define-by-Run 및 동적 탐색 공간**
코드가 실행되는 도중에 상황에 맞춰 탐색 공간을 유연하게 바꿀 수 있음
- **가지치기 기능**
학습 과정에서 성능이 좋지 않은 trial을 조기에 종료하여 불필요한 자원 낭비를 줄임
- **경량성과 확장성**
가볍고 설치가 쉬우며, 로컬 환경에서부터 분산 환경에까지 다양한 규모의 최적화 작업에 유연하게 적용 가능함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }
```

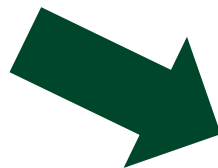
- loss_function: 모델이 학습에서 틀렸을 때 얼마나 벌점을 줄지 계산하는 공식

3-2. Optuna 하이퍼파라미터 튜닝

```
print("타겟 클래스 종류:", train_df['credit'].unique())
```

✓ 0.0s

타겟 클래스 종류: [1. 2. 0.]



```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",
```

- loss_function: 모델이 학습에서 틀렸을 때 얼마나 벌점을 줄지 계산하는 공식
- 현재 타겟 데이터의 클래스가 0, 1, 2로 3개이므로, MultiClass전용 손실 함수를 사용하여, 다중 분류에 최적화된 학습을 진행함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }
```

- eval_metric: 학습 중간중간 '모델이 얼마나 잘하고 있는지' 검사하는 기준
- 보통 loss_function과 일치시켜 학습의 일관성을 유지함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }
```

- iterations: 모델이 데이터를 최대 몇 번까지 반복해서 학습할지 결정하는 수치
- early_stopping_rounds과 함께 사용하여 성능 개선이 없으면 중간에 멈추게 함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }
```

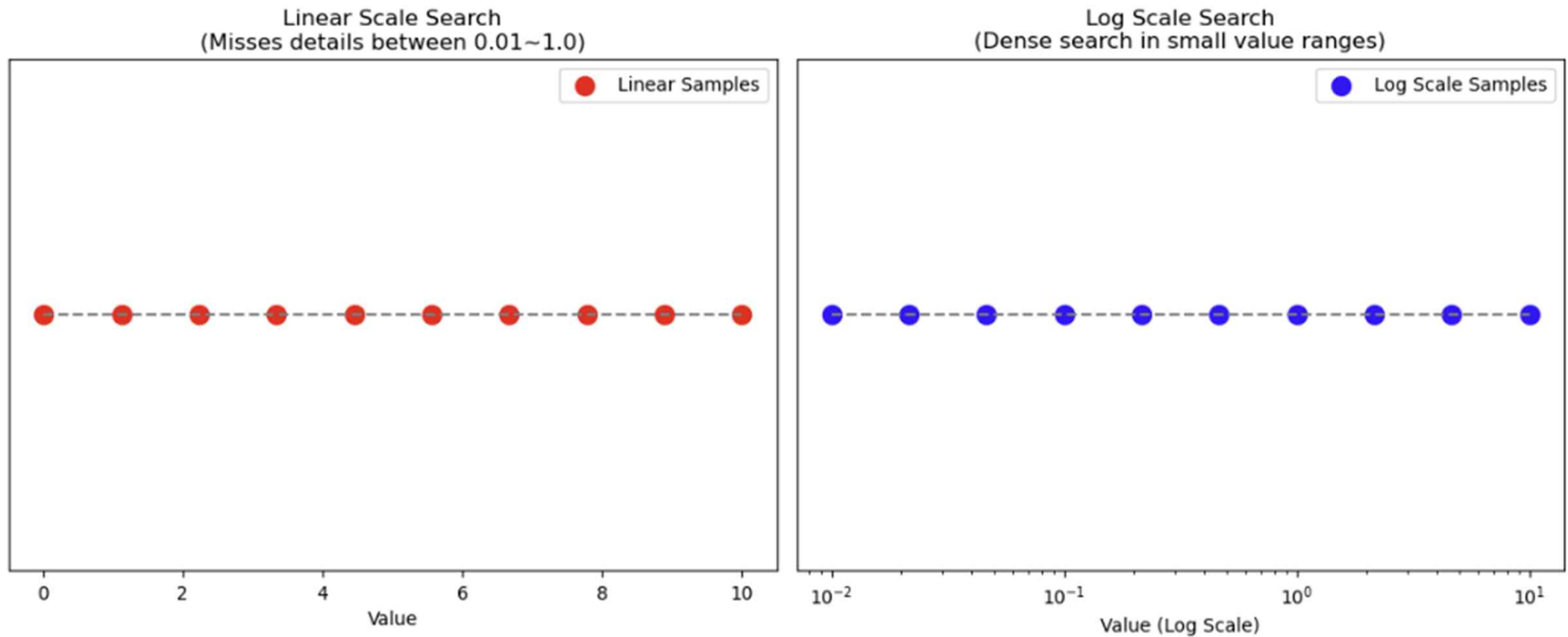
- learning_rate: 경사하강법에서 가중치가 한 번에 얼마나 크게 이동할지를 정함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }
```

- depth: 과적합을 방지하면서도 최적의 성능을 낼 수 있는 범위를 지정함

3-2. Optuna 하이퍼파라미터 튜닝



- 로그 스케일 사용 이유: 값이 작을수록 변화에 민감한 하이퍼파라미터의 특성을 반영하여, 정밀하게 탐색함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }
```

- l2_leaf_reg(L2 정규화/규제): 손실 함수에 L2 규제 항을 추가하여 모델의 가중치가 너무 커지지 않도록 억제함
- 너무 극단적인 예측을 방지하여 모델의 안정성을 높이기 위해 로그를 사용함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }
```

- random_strength(랜덤 강도): 트리의 분할 지점(split)을 선택할 때 점수에 무작위 노이즈를 더하는 강도
- CatBoost가 매 단계 선택을 할 때 약간의 '랜덤성'을 섞어줌으로써 모델이 매번 똑같은 패턴만 찾는 것을 방지함

3-2. Optuna 하이퍼파라미터 튜닝

```
def objective(trial):  
    param = {  
        "loss_function": "MultiClass",  
        "eval_metric": "MultiClass",  
        "iterations": 1000,  
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1, log=True),  
        "depth": trial.suggest_int("depth", 4, 10),  
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1e-2, 10.0, log=True),  
        "random_strength": trial.suggest_float("random_strength", 1.0, 10.0),  
        "bagging_temperature": trial.suggest_float("bagging_temperature", 0.0, 1.0),  
        "random_seed": 42,  
        "verbose": False,  
        "allow_writing_files": False  
    }  
}
```

- bagging_temperature(배깅 온도): 베이저안 배깅(Bayesian Bagging) 강도를 조절함
- 0이면 모든 샘플에 동일한 가중치를 주어 일반적인 학습을 하고, 1에 가까워질수록 샘플 가중치의 변동폭이 커짐
- 온도를 적절히 높이면 학습 시 데이터 샘플링에 변주를 주어, 과적합을 방지하는 효과가 있음

3-2. Optuna 하이퍼파라미터 튜닝

```
skf_tune = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
cv_scores = []

for tr_idx, va_idx in skf_tune.split(X_train_cb, y_train_cb):
    X_tr, X_va = X_train_cb.iloc[tr_idx], X_train_cb.iloc[va_idx]
    y_tr, y_va = y_train_cb.iloc[tr_idx], y_train_cb.iloc[va_idx]

    train_pool = Pool(X_tr, y_tr, cat_features=cat_cols)
    valid_pool = Pool(X_va, y_va, cat_features=cat_cols)

    model = CatBoostClassifier(**param)
    model.fit(train_pool, eval_set=valid_pool, early_stopping_rounds=50, use_best_model=True)

    preds = model.predict_proba(valid_pool)
    cv_scores.append(log_loss(y_va, preds))

return np.mean(cv_scores)
```

- StratifiedKFold: 클래스 비율을 유지하며 데이터를 3개로 쪼개어 검증하는 과정을 통해, 유연하게 대처하는 강건한 모델인지 검증함

3-2. Optuna 하이퍼파라미터 튜닝

```
skf_tune = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
cv_scores = []

for tr_idx, va_idx in skf_tune.split(X_train_cb, y_train_cb):
    X_tr, X_va = X_train_cb.iloc[tr_idx], X_train_cb.iloc[va_idx]
    y_tr, y_va = y_train_cb.iloc[tr_idx], y_train_cb.iloc[va_idx]

    train_pool = Pool(X_tr, y_tr, cat_features=cat_cols)
    valid_pool = Pool(X_va, y_va, cat_features=cat_cols)

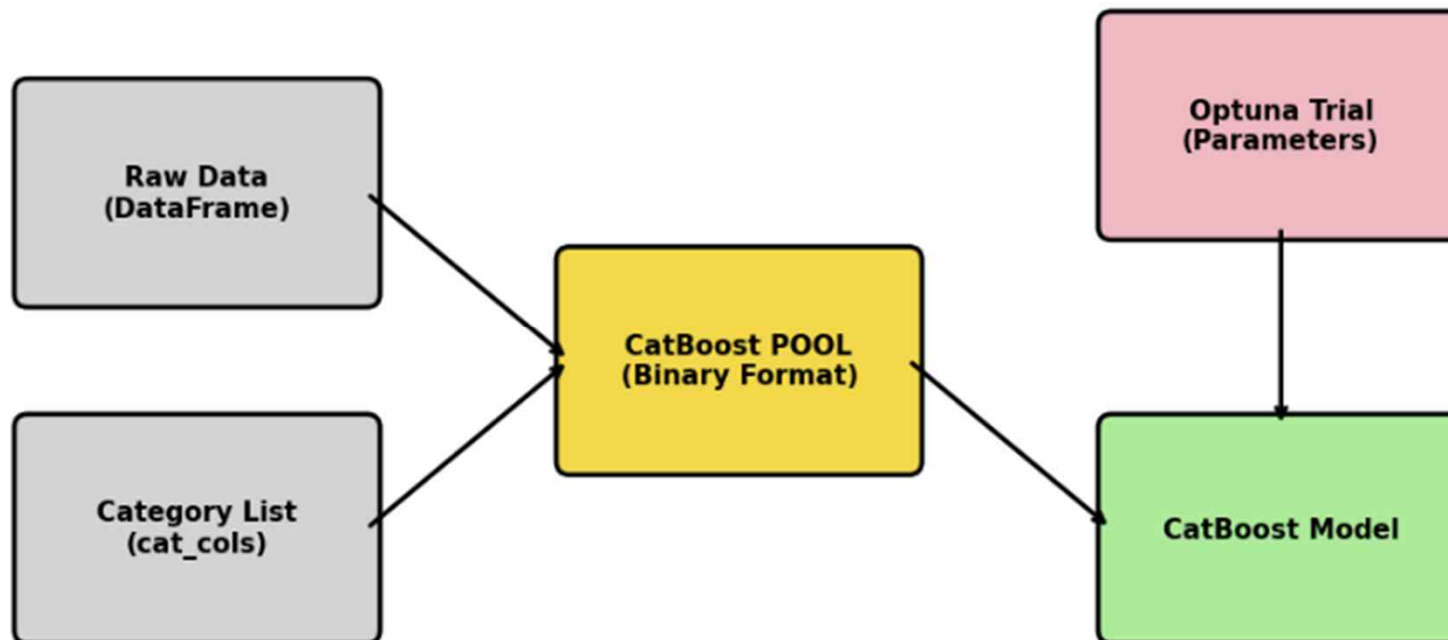
    model = CatBoostClassifier(**param)
    model.fit(train_pool, eval_set=valid_pool, early_stopping_rounds=50, use_best_model=True)

    preds = model.predict_proba(valid_pool)
    cv_scores.append(log_loss(y_va, preds))

return np.mean(cv_scores)
```

- Pool은 CatBoost 엔진이 읽기 가장 좋은 바이너리 형태로 데이터를 압축하여 들고 있음으로써 메모리 부족 문제를 해결하고, 학습 시간을 단축해줌

3-2. Optuna 하이퍼파라미터 튜닝



- Pool은 CatBoost 엔진이 읽기 가장 좋은 바이너리 형태로 데이터를 압축하여 들고 있음으로써 메모리 부족 문제를 해결하고, 학습 시간을 단축해줌

3-2. Optuna 하이퍼파라미터 튜닝

```
skf_tune = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
cv_scores = []

for tr_idx, va_idx in skf_tune.split(X_train_cb, y_train_cb):
    X_tr, X_va = X_train_cb.iloc[tr_idx], X_train_cb.iloc[va_idx]
    y_tr, y_va = y_train_cb.iloc[tr_idx], y_train_cb.iloc[va_idx]

    train_pool = Pool(X_tr, y_tr, cat_features=cat_cols)
    valid_pool = Pool(X_va, y_va, cat_features=cat_cols)

    model = CatBoostClassifier(**param)
    model.fit(train_pool, eval_set=valid_pool, early_stopping_rounds=50, use_best_model=True)

    preds = model.predict_proba(valid_pool)
    cv_scores.append(log_loss(y_va, preds))

return np.mean(cv_scores)

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=20)
best_params = study.best_params
print(best_params)
```

- study.optimize를 통해 총 20번의 실험을 수행함

3-3. 평가지표: Logloss

$$\text{multiclass logloss} = -\frac{1}{N} \sum_{i=1}^N \sum_{m=1}^M y_{i,m} \log p_{i,m}$$

다중 분류 로그 손실 계산 수식

M : 클래스 수

$y_{i,m}$: 행 데이터 i 가 클래스 m 에 속할 경우 1, 그렇지 않을 경우 0

$p_{i,m}$: 행 데이터 i 가 클래스 m 에 속하는 예측 확률



각 class별 예측 확률을 제출하는 방식
확신을 가지고 틀릴 경우 손실 매우 커짐

3-4. 5-fold 7-seed ensemble

- 안정적인 예측을 위해 5-fold로
7개의 Seed 앙상블
- seeds = [0, 1, 2, 42, 77, 202, 777]

```
oof_final = np.zeros((len(X_train_cb), n_class))
test_final = np.zeros((len(X_test_cb), n_class))

for seed in seeds:
    print(f"\n===== Seed {seed} =====")
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

    oof_seed = np.zeros((len(X_train_cb), n_class))
    test_seed = np.zeros((len(X_test_cb), n_class))

    for fold, (tr_idx, va_idx) in enumerate(skf.split(X_train_cb, y_train_cb), 1):
        X_tr, y_tr = X_train_cb.iloc[tr_idx], y_train_cb.iloc[tr_idx]
        X_va, y_va = X_train_cb.iloc[va_idx], y_train_cb.iloc[va_idx]

        train_pool = Pool(X_tr, y_tr, cat_features=cat_cols)
        valid_pool = Pool(X_va, y_va, cat_features=cat_cols)
        test_pool = Pool(X_test_cb, cat_features=cat_cols)

        model = CatBoostClassifier(
            loss_function="MultiClass",
            eval_metric="MultiClass",
            iterations=5000,
            random_seed=seed,
            verbose=200,
            **best_params
        )

        model.fit(train_pool, eval_set=valid_pool, early_stopping_rounds=100, use_best_model=True)

        oof_seed[va_idx] = model.predict_proba(valid_pool)
        test_seed += model.predict_proba(test_pool) / n_splits
```

3-4. 5-fold 7-seed ensemble

```
oof_final = np.zeros((len(X_train_cb), n_class))
test_final = np.zeros((len(X_test_cb), n_class))

for seed in seeds:
    print(f"\n===== Seed {seed} =====")
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

    oof_seed = np.zeros((len(X_train_cb), n_class))
    test_seed = np.zeros((len(X_test_cb), n_class))

    for fold, (tr_idx, va_idx) in enumerate(skf.split(X_train_cb, y_train_cb), 1):
        X_tr, y_tr = X_train_cb.iloc[tr_idx], y_train_cb.iloc[tr_idx]
        X_va, y_va = X_train_cb.iloc[va_idx], y_train_cb.iloc[va_idx]

        train_pool = Pool(X_tr, y_tr, cat_features=cat_cols)
        valid_pool = Pool(X_va, y_va, cat_features=cat_cols)
        test_pool = Pool(X_test_cb, cat_features=cat_cols)

        model = CatBoostClassifier(
            loss_function="MultiClass",
            eval_metric="MultiClass",
            iterations=5000,
            random_seed=seed,
            verbose=200,
            **best_params
        )

        model.fit(train_pool, eval_set=valid_pool, early_stopping_rounds=100, use_best_model=True)

        oof_seed[va_idx] = model.predict_proba(valid_pool)
        test_seed += model.predict_proba(test_pool) / n_splits
```

Oof_seed: 검증폴드 예측값 합치기
Test_seed: 폴드별 테스트 데이터 예측값 평균

3-4. 5-fold 7-seed ensemble

```
oof_final += oof_seed / len(seeds)
test_final += test_seed / len(seeds)
```

```
# =====
# [STEP 3] 제출 저장 (경로 수정 필수!)
# =====
cat_submission = submission.copy()
cat_submission["0"] = test_final[:, 0]
cat_submission["1"] = test_final[:, 1]
cat_submission["2"] = test_final[:, 2]
```

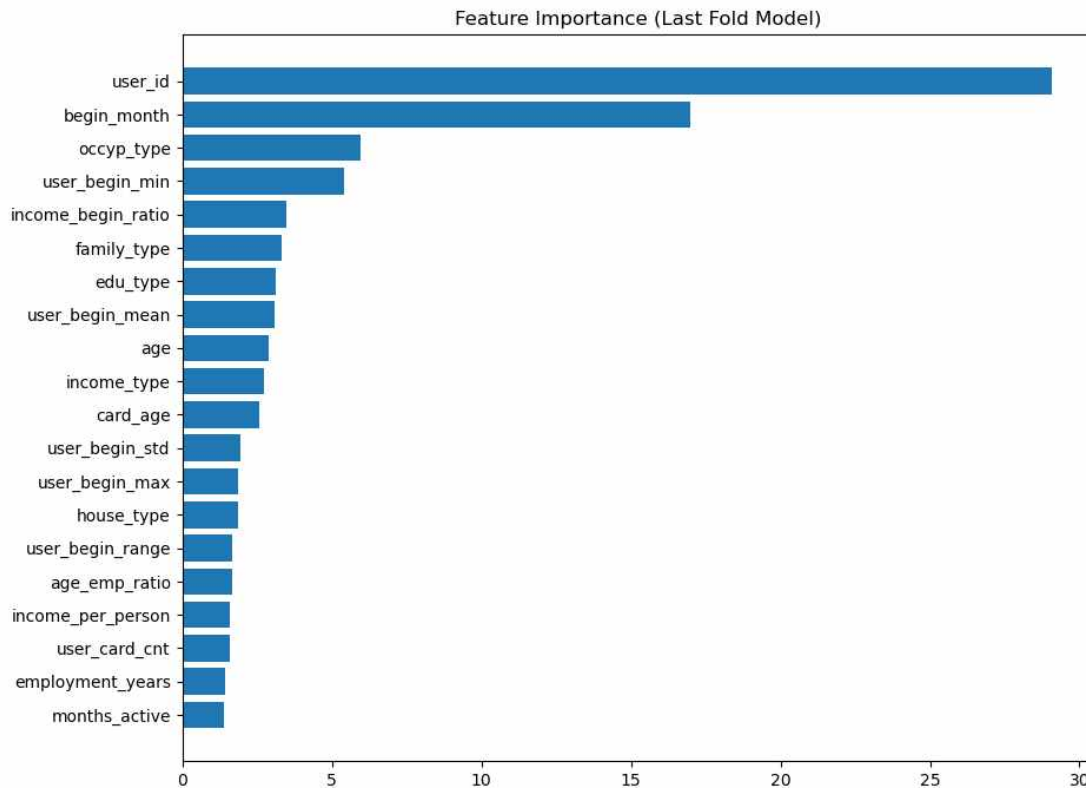
Oof_final: 최종 logloss 계산용

Test_final: seed별 예측 확률 평균하여
최종 예측 확률 생성

```
===== FINAL (Optuna best + seed7) =====
Final OOF: 0.6624934141939367
test_final shape: (10000, 3)
row sum check: 1.0
✅ saved: C:\Users\woori\Desktop\ESAA\winter_vacation\changed_project\re
index      0      1      2
0  26457  0.097614  0.142085  0.760302
1  26458  0.296983  0.246284  0.456733
2  26459  0.040781  0.072731  0.886488
3  26460  0.056779  0.074804  0.868417
4  26461  0.078470  0.250565  0.670966
```

제출파일 head

3-5. feature importance



Id변수의 높은 중요도:
동일한 고객을 잘 예측하여 정확도 높임
=>캣부스트 고성능 원인

4. 제출 결과

제출 일시	public점수 private점수
2026-01-29 10:01:07	0.6675784715 0.6609302969

Public 기준 상위 10%
Private 기준 상위 1%

Q&A